

# Informe PC4 - Desarrollo de Software

**Curso:** CC3S2 Desarrollo de Software **Alumna:** Melissa Iman Noriega **Fecha:** 01/07/2026 **Repositorio:** <https://github.com/Melissa1221/pc4-desarrollo-software>

## 1. Descripción

Se desarrolló el núcleo de dominio en Java (Maven) de una plataforma de rescate y adopción de mascotas. El trabajo cubre los tres módulos del enunciado y se centra en los patrones de diseño de comportamiento, que fue lo que se pidió priorizar. Según la indicación, no se implementaron las búsquedas por especialidad: solo se desarrolló la interfaz de cada módulo siguiendo los patrones. Los datos se manejan en memoria.

## 2. Requisitos cumplidos

### Módulo 1 — Reporte de animales perdidos y alertas

- RF 1.1: se registra una mascota perdida con nombre, especie, raza, foto y descripción (LostPet).
- RF 1.2: el reporte guarda las coordenadas geográficas.
- RF 1.3: un ciudadano anónimo registra un avistamiento con foto y ubicación (Sighting).
- RF 1.4: al reportar una mascota se notifica a los suscriptores dentro de un radio configurable (AlertCenter).
- RNF 1.2 (anonimato): el id del dueño no se expone en la alerta que reciben los ciudadanos.

### Módulo 2 — Buscador por imagen

- RF 2.1: se carga una imagen para la búsqueda (ImageQuery).
- RF 2.2: la búsqueda se resuelve según la intención elegida (adopción, venta, verificar pérdida).
- RF 2.3 / 2.4 / 2.5: cada intención tiene su propia clase; la búsqueda concreta se deja como interfaz, sin implementar el motor.
- RNF 2.1 (abstracción): los metadatos van en un mapa clave-valor, de modo que el motor es intercambiable.

### Módulo 3 — Red de cuidadores

- RF 3.1: el perfil guarda el rol del cuidador (solidario, profesional, especializado).
- RF 3.2: el cuidador define sus restricciones (especies aceptadas).
- RF 3.3: interruptor para activar/desactivar las alertas del módulo 1.
- RNF 3.1 (confiabilidad): el perfil solo se habilita si pasa la validación del documento.

## 3. Patrones de diseño de comportamiento usados

Se eligieron seis patrones de comportamiento, dos por módulo. Todos los paths que se mencionan cuelgan del paquete base `com.uni.petfinder`.

### 3.1 Observer (módulo 1)

El Observer sirve para que varios objetos se enteren de un cambio sin que quien lo provoca tenga que conocerlos. Es justo lo que necesita el RF 1.4: cuando alguien reporta una mascota perdida, hay que avisar a los cuidadores cercanos, que van cambiando todo el tiempo (unos se conectan, otros se van).

El *subject* es AlertCenter y los suscriptores implementan la interfaz AlertObserver.

- `alertas/AlertCenter.java`
- `alertas/AlertObserver.java`

```
public void reportLostPet(LostPet pet, double radiusKm) {
    for (Subscription s : subs) {
        if (distanceKm(pet.getLat(), pet.getLon(), s.lat, s.lon) <= radiusKm) {
            s.obs.onLostPet(pet);
        }
    }
}
```

Un caso de uso concreto: Ana vive a 200 m de donde se perdió *Firulais* y Luis a 20 km. Ambos están suscritos, pero al reportar con radio de 1 km solo Ana recibe el aviso. Si mañana se agrega un canal nuevo (por ejemplo, mandar la alerta por correo), basta con crear otro *AlertObserver* y suscribirlo; el *AlertCenter* no cambia.

### 3.2 State (módulo 1)

Un reporte no es siempre lo mismo: primero está activo, luego la mascota aparece y al final el caso se cierra. En vez de llenar *LostPet* de *if* para saber en qué momento está, cada estado se convierte en una clase que sabe cómo comportarse y a qué estado pasar.

- alertas/ReportState.java (interfaz)
- alertas/ActiveState.java, FoundState.java, ClosedState.java
- alertas/LostPet.java (mantiene el estado actual)

```
public class ActiveState implements ReportState {
    @Override public String name() { return "ACTIVO"; }
    @Override public ReportState next() { return new FoundState(); }
}
```

Caso de uso: el dueño reporta a *Firulais* (estado ACTIVO). Cuando un vecino lo encuentra, el dueño toca “marcar como encontrado” y *LostPet* solo llama a *advance()*, que internamente hace *state = state.next()* y pasa a ENCONTRADO. Cada estado decide su transición, así que agregar un estado intermedio (por ejemplo, “en revisión”) no obliga a tocar los demás.

### 3.3 Strategy (módulo 2)

El buscador hace lo mismo a alto nivel —recibe una imagen y devuelve resultados— pero la lógica cambia según lo que quiera el usuario. *Strategy* permite tener esa familia de algoritmos separada y elegir uno en tiempo de ejecución, que es lo que pide el RF 2.2 (elegir entre adopción, venta o verificar pérdida) y el RNF 2.1 (motor intercambiable).

- busqueda/SearchStrategy.java (interfaz)
- busqueda/AdoptionSearch.java, SaleSearch.java, LostCheckSearch.java

```
public interface SearchStrategy {
    List<String> run(ImageQuery query);
}
```

Caso de uso: el usuario sube la foto de un perro y marca “Adopción”. El sistema usa la estrategia *AdoptionSearch*, que solo mira el catálogo de protectoras. Si en cambio marca “Venta”, se usa *SaleSearch* sin cambiar nada más. El día que se quiera conectar un motor real de reconocimiento de imágenes, se reemplaza la estrategia y el resto del sistema ni se entera.

### 3.4 Template Method (módulo 2)

Todas las búsquedas siguen los mismos pasos: validar que llegó la imagen, buscar y dar formato al resultado. Lo único que cambia es *cómo* busca cada intención. *Template Method* deja fijo ese orden en un

método y solo pide a cada subclase que complete el paso de la búsqueda.

- `busqueda/SearchTemplate.java`

```
public final List<String> run(ImageQuery query) {
    validate(query);
    List<String> results = search(query);
    return format(results);
}

protected abstract List<String> search(ImageQuery query);
```

Caso de uso: `AdoptionSearch` y `SaleSearch` no repiten el `validate()` ni el `format()`; solo escriben su `search()`. Si más adelante se decide validar también el tamaño del archivo, se agrega una vez en `SearchTemplate` y aplica a las tres intenciones de golpe.

### 3.5 Command (módulo 3)

El RF 3.3 pide un interruptor para prender y apagar las alertas de un cuidador. Con Command, cada acción (activar, desactivar) se vuelve un objeto propio, y el interruptor solo se encarga de dispararla sin saber qué hace por dentro.

- `cuidadores/AlertCommand.java` (interfaz)
- `cuidadores/EnableAlertsCommand.java`, `DisableAlertsCommand.java`
- `cuidadores/AlertSwitch.java` (el interruptor)

```
public class AlertSwitch {
    public void press(AlertCommand command, CaregiverProfile profile) {
        command.execute(profile);
    }
}
```

Caso de uso: María se va de vacaciones y desactiva las alertas. La app llama a `aSwitch.press(new DisableAlertsCommand(), maria)`. Si después se quiere agregar una acción como “silenciar por 2 horas”, se crea un comando nuevo y el interruptor queda igual.

### 3.6 Chain of Responsibility (módulo 3)

Antes de mostrar un perfil de cuidador al público hay que revisar varias cosas (RNF 3.1). En vez de un método gigante con todas las validaciones juntas, cada revisión es un eslabón: si pasa, deja seguir al siguiente; si falla, corta la cadena.

- `cuidadores/ProfileValidator.java` (base de la cadena)
- `cuidadores/NameValidator.java`, `IdValidator.java`

```
public boolean validate(CaregiverProfile profile) {
    if (!check(profile)) {
        return false;
    }
    if (next == null) {
        return true;
    }
    return next.validate(profile);
}
```

Caso de uso: se arma la cadena `NameValidator` → `IdValidator`. Si un cuidador tiene nombre pero no subió su documento, `IdValidator` corta y el perfil no se habilita. Añadir una nueva regla (por ejemplo, verificar el teléfono) es crear otro validador y encadenarlo, sin tocar los que ya existen.

## 4. Arquitectura

El código está separado por módulo en tres paquetes independientes:

com.uni.petfinder

|              |                                                |
|--------------|------------------------------------------------|
| — alertas    | → módulo 1 (Observer + State)                  |
| — busqueda   | → módulo 2 (Strategy + Template Method)        |
| — cuidadores | → módulo 3 (Command + Chain of Responsibility) |
| — App        | → demostración de los tres módulos             |

La separación en paquetes responde al RNF 3.2 (escalabilidad): los módulos no dependen entre sí, así que un pico de demanda en uno no afecta al otro. La comunicación es por interfaces, lo que permite reemplazar implementaciones sin tocar el resto.

## 5. Cómo ejecutar

```
mvn compile
mvn exec:java
```

La clase App ejecuta los tres módulos e imprime el resultado de cada patrón.